From Cloud Build to Running Hardware

Development specification: build host, relay, and Kria KV260 target
for the hardware rendering of the rho calculus

F1R3FLY.io design note — draft 0.1

13 November 2026

Abstract

This note specifies the path a build takes from source to running hardware for the rho hardware rendering: a rented cloud instance builds the bitstream, an iMac relays it and serves as the terminal, and an AMD Kria KV260 runs it. It fixes the roles of the three machines, the artifacts that cross between them, the board-support wrapper that the tile array needs on this board, the packaging and loading procedure, the checks that establish that the hardware under test is the hardware that was built, and a bring-up ladder. It is a companion to the design specification (draft 0.5) and changes nothing in it. Nothing here has yet been run on a board; every decision is marked proposed, and Section 13 lists what the first bring-up has to settle.

Contents

1 Purpose and scope **2**

2 Roles and topology **2**

3 Decision register **3**

4 Artifacts **3**

5 Build host **4**
5.1 Instance 4
5.2 Tools 4
5.3 Repository layout 4
5.4 Build steps 4

6 Board-support wrapper **5**
6.1 Clock, reset, stall 5
6.2 Register map 6

7 Packaging **6**

8 Transfer **7**

9 Target **7**
9.1 One-time setup 7
9.2 Load 8
9.3 Target tools 8
9.4 Logs back 8

10 Verification of the path 8

11 Bring-up ladder 9

12 Failure modes 9

13 Open questions 9

14 Sources 10

1 Purpose and scope

The design specification says what the machine is. This note says how a change to the Bluespec source becomes a running array of tiles on a board, and how the evidence of what the array did gets back to the checker. Three constraints shape it.

- The vendor tools for this board run on x86-64 Linux or Windows and not on macOS, so builds happen on a rented cloud instance.
- The board sits on a local network behind a router, so the cloud instance cannot reach it. The iMac can reach both.
- The verification obligations of the design specification are stated over decision logs. A log is evidence only if the bitstream that produced it is known to be the one that was built from the source under review.

In scope: the three machines and their roles, the artifacts, the board-support wrapper, packaging, transfer, loading, log return, and the checks on the path itself. Out of scope: the machine (design specification), the f1r3lang compiler, and timing closure for any particular array size.

2 Roles and topology

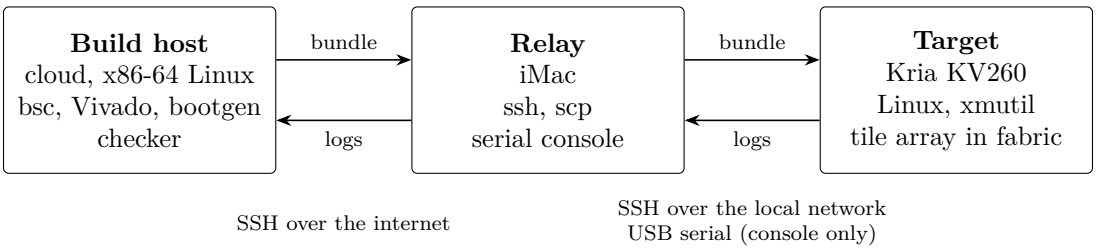


Figure 1: The three machines. The iMac opens both connections; neither end connects to the other.

Machine	Does	Does not
Build host	Compiles Bluespec to Verilog, simulates, synthesises, places and routes, packages the bundle, runs the checker over returned logs	Reach the board
Relay (iMac)	Starts builds over SSH, copies the bundle across, opens the shell and the serial console on the board, flashes the SD card once	Run any vendor tool; drive JTAG
Target (Kria)	Loads the bundle into the fabric at run time, loads images, runs, drains decision logs to files	Build anything

Because the relay runs no vendor tool, its processor architecture does not matter. The question of Intel or Apple silicon, open in the board discussion, is closed by this arrangement.

3 Decision register

All decisions are proposed by this draft. They are numbered B1 onward to keep them apart from D1 to D32 of the design specification.

	Decision
B1	Builds run on a cloud x86-64 Linux instance, started for a build and stopped afterwards.
B2	The bitstream is loaded at run time from the board's own Linux with `xmutil`; no JTAG cable and no boot-image rebuild is part of the normal loop.
B3	The relay pulls from the build host and pushes to the target. No inbound connection to the local network is opened. An overlay network (Section 8) is an allowed alternative, not the default.
B4	The tile array is wrapped in a board-support module with one AXI4-Lite slave, clocked and reset by the processor block. The wrapper is the only board-specific RTL.
B5	The wrapper is written in Bluespec, like the rest of the design, with no external AXI library.
B6	Every bitstream carries a build identity (source revision, build time, array dimensions) readable from the register interface.
B7	Every bundle carries a manifest of SHA-256 digests, checked on the target before loading.
B8	When the log buffer is nearly full the array stalls by clock enable; it never drops a log record.
B9	Log readout is by register access for stages L1 to L3 (Section 11); a streaming path replaces it only when measured throughput requires.
B10	The device tree overlay is generated from the hardware description exported by the build, not written by hand.

4 Artifacts

A *bundle* is a directory named for the application; the name used throughout is `rho`. The loader on the target requires three files: the bitstream in `.bit.bin` form, a device tree overlay, and a `shell.json` descriptor. The bitstream and the overlay share a base name. This note adds a fourth file, the manifest.

File	Made by	Content
`rho.bit.bin`	Vivado, then bootgen	The fabric configuration with the bitstream header stripped
`rho.dtbo`	device tree generator, then `dtc`	Overlay naming the bitstream, enabling the fabric clock, and declaring the register region
`shell.json`	fixed text	Descriptor for the loader: a flat design with one slot
`MANIFEST`	build script	Build identity and the SHA-256 digest of each of the other three files

Images (compiled programs for the array, in the image format of the design specification) and decision logs are not part of the bundle. Images travel the same route in the same direction; logs travel it in reverse.

5 Build host

5.1 Instance

An x86-64 Linux instance on a distribution the installed Vivado version supports. A starting size of 8 virtual CPUs, 32 GB of memory and 150 GB of disk is proposed; the disk is dominated by the Vivado installation. The device on the Kria K26 module is in the set that the no-cost Vivado edition covers, to be confirmed against the licence terms of the version installed. Inbound access is SSH only, by key, restricted to the address of the relay. The instance is stopped when no build is running; its disk persists.

5.2 Tools

- `bsc`, the Bluespec compiler, release 2026.07.1 (the version already exercised for this project).
- Icarus Verilog, for simulation of the generated Verilog before any synthesis is attempted.
- Vivado, with `bootgen` and the device tree generator that ship with it.
- `dtc`, the device tree compiler.
- GHC, to build `rhoref` and its checker.

5.3 Repository layout

```
rho-hw/
  bsv/            Bluespec source: tile, mesh, array, board wrapper
  rhoref/         reference implementation and checker
  board/kv260/    build.tcl, shell.json
  scripts/        build.sh, package.sh
  build/          generated; never committed
    verilog/      output of bsc
    vivado/       project, reports, rho.bit, system.xsa
    bundle/rho/   the four files of Section 4
```

5.4 Build steps

1. **Identity.** Record the source revision and the build time; refuse to produce a release bundle from a working tree with uncommitted changes.

2. **Compile.** Run `bsc -verilog` on the board wrapper as the top module, with the array dimensions and the identity as parameters.
3. **Simulate.** Run the generated Verilog under Icarus against the directed programs of the design specification and compare logs with `rhoref`. A failure here stops the build; synthesis time is not spent on a design that fails simulation.
4. **Synthesise and implement.** Run Vivado in batch mode on `build.tcl`. Sources are the generated Verilog and the Verilog primitives that `bsc` ships for Vivado. A build that does not meet timing fails; no bundle is produced.
5. **Export.** Write `rho.bit` and the hardware description `system.xsa`. Keep the utilisation and timing reports beside them.
6. **Package.** Produce the bundle (Section 7).

The Tcl below is a sketch of step 4, to be made exact against the installed Vivado version at first bring-up.

```
create_project rho_kv260 build/vivado -part xck26-sfvc784-2LV-c
set_property board_part [get_board_parts *kv260_som*] [current_project]
add_files [glob build/verilog/*.v]
create_bd_design system
create_bd_cell -type ip -vlnv xilinx.com:ip:zynq_ultra_ps_e ps
apply_bd_automation -rule xilinx.com:bd_rule:zynq_ultra_ps_e \
    -config {apply_board_preset 1} [get_bd_cells ps]
create_bd_cell -type module -reference mkRhoBoard rho
# connect the AXI4-Lite slave to the processor block, with clock and reset
apply_bd_automation -rule xilinx.com:bd_rule:axi4 \
    -config {Master /ps/M_AXI_HPM0_FPD Clk Auto} [get_bd_intf_pins rho/s_axi]
make_wrapper -files [get_files system.bd] -top -import
launch_runs impl_1 -to_step write_bitstream -jobs 8
wait_on_run impl_1
write_hw_platform -fixed -include_bit build/vivado/system.xsa
```

6 Board-support wrapper

On this board the fabric has no oscillator of its own in the path used here: clock and reset come from the processor block. The top level is therefore not the tile array but a wrapper, `mkRhoBoard`, containing the array and one AXI4-Lite slave.

6.1 Clock, reset, stall

- One clock domain, driven by the first fabric clock of the processor block. Its frequency is a build parameter, chosen after the first tile has been synthesised.
- Reset is the processor block's fabric reset, synchronised in the wrapper. A second, soft reset is a bit of the control register.
- The array has a clock enable. It is deasserted while the run bit is clear and while the log buffer is above its high-water mark (B8). The cycle counter of the design specification counts enabled cycles, so a stall is invisible to the logs and the replay obligation is unaffected.

6.2 Register map

Registers are 32 bits wide. Offsets are from the base address that Vivado assigns to the slave, which the overlay records. Wider machine words cross as consecutive 32-bit accesses, low half first.

Offset	Name	Access	Meaning
0x00	ID	read	Constant 0x52484F31
0x04	BUILD_REV	read	Source revision, first 32 bits
0x08	BUILD_TIME	read	Build time, seconds since the epoch
0x0C	CONFIG	read	Array width, array height, specification draft
0x10	CTRL	write	Run, soft reset, single step
0x14	STATUS	read	Running, quiescent, fault, stalled
0x18	CYCLE_LO	read	Enabled cycles, low half
0x1C	CYCLE_HI	read	Enabled cycles, high half
0x20	TILE_SEL	write	Tile addressed by the seed and image registers
0x24	SEED	write	Arbiter seeds of the selected tile, in order
0x28	IMG_ADDR	write	Image load address in the selected tile
0x2C	IMG_DATA	write	Image word; the address then advances
0x30	INGRESS	write	Term-stream word into the array boundary
0x34	EGRESS	read	Term-stream word out of the boundary; a read removes it
0x38	IO_LEVEL	read	Free ingress space, pending egress words
0x3C	LOG_DATA	read	Next log word; a read removes it
0x40	LOG_LEVEL	read	Log words pending
0x44	FAULT_TILE	read	Tile that raised the first fault
0x48	FAULT_CODE	read	Fault code of the design specification
0x4C	SCRATCH	read, write	No effect; used to test the bus

The static code segment is replicated in every tile, so image loading writes the same words to each tile in turn; a broadcast value of TILE_SEL is reserved for that purpose. Image and seed writes are accepted only while the run bit is clear.

7 Packaging

Bitstream. Strip the header with bootgen:

```
echo 'all:{rho.bit}' > bootgen.bif
bootgen -w -arch zynqmp -process_bitstream bin -image bootgen.bif
```

The result is rho.bit.bin.

Overlay. Generate the fabric device tree source from system.xsa with the device tree generator of the installed tool version. In the generated source, set the firmware name to the name of the packaged bitstream, then compile:

```
firmware-name = "rho.bit.bin";

dtc -@ -O dtb -o rho.dtbo pl.dtsi
```

The generated overlay also enables the fabric clock and declares the register region, which is why it is generated and not written by hand (B10). The register region is bound to the generic user-space I/O driver, so the tools on the target reach it without a kernel module.

Descriptor. The same text for every build:

```
{
  "shell_type": "XRT_FLAT",
  "num_slots": "1"
}
```

Manifest. One line of build identity (revision, time, width, height), then one line per file with its SHA-256 digest, in the format `sha256sum` checks. The identity line repeats what the `BUILD_REV`, `BUILD_TIME` and `CONFIG` registers will report.

8 Transfer

The relay holds two host entries in its SSH configuration, one for each end, each with its own key.

```
Host cloud
  HostName <address of the build host>
  User <build user>
  IdentityFile ~/.ssh/rho_cloud
Host kria
  HostName <address of the board on the local network>
  User <board user>
  IdentityFile ~/.ssh/rho_kria
```

One command on the relay copies the bundle from one end to the other through the relay, without leaving a copy on it:

```
scp -3 -r cloud:rho-hw/build/bundle/rho kria:incoming/
```

A script, `deploy.sh`, wraps the copy and the load of Section 9.2, so the normal loop from the relay is two commands: one that starts the build on the build host and one that deploys.

Allowed alternative. Joining the build host and the board to an overlay network lets the build host deploy directly and removes the relay from the loop. It is not the default because it places the board on a network reachable from a rented machine.

9 Target

9.1 One-time setup

1. From the relay, write the board's Linux image to a microSD card with a card-flashing utility that runs on macOS.
2. Connect the board to the local network by Ethernet and to the relay by the micro-USB port, which carries the serial console (115200 baud, 8 data bits, no parity, 1 stop bit). On the relay a terminal program opens the serial device; no driver is expected to be needed.
3. Boot, log in on the console, change the default password, read the board's network address, install the relay's public key, and confirm SSH from the relay. After this the serial console is needed only when the network is not.
4. Confirm that `xmutil` is present and that the boot firmware on the module is recent enough for the image; update it if the image's release notes require.

5. Install the target tools (Section 9.3).

9.2 Load

```
cd incoming/rho && tail -n +2 MANIFEST | sha256sum -c -
sudo mkdir -p /lib/firmware/xilinx/rho
sudo cp rho.bit.bin rho.dtbo shell.json /lib/firmware/xilinx/rho/
sudo xmutil listapps
sudo xmutil unloadapp
sudo xmutil loadapp rho
```

Loading reconfigures the fabric without a reboot. The load is accepted only after the identity check of obligation P2 passes.

9.3 Target tools

A single small program, `rhoctl`, built on the target from C source in the repository, maps the register region and offers:

- `rhoctl id`: print the four identity registers and compare them with a manifest.
- `rhoctl load IMAGE`: write seeds and an image, with the run bit clear.
- `rhoctl run [-cycles N] -log FILE`: set the run bit, drain `LOG_DATA` to a file until quiescence, a fault, or the cycle bound, and record the final status and cycle count.
- `rhoctl send` and `rhoctl recv`: move term-stream packets through the ingress and egress registers.

9.4 Logs back

Log files are copied from the target to the build host by the reverse of the transfer command. The checker reads the log together with the image and the seeds that produced it, all three named by digest in a run record written by `rhoctl run`.

10 Verification of the path

The obligations of the design specification are about the machine. The obligations here are about the path, and exist so that a passing check on a log means what it appears to mean.

Obligation P1 (Integrity)**.** The files loaded on the target have the digests recorded in the manifest produced by the build. Discharged by the digest check before every load.

Obligation P2 (Identity)**.** After a load, `ID` reads the constant, and `BUILD_REV`, `BUILD_TIME` and `CONFIG` equal the identity line of the manifest. Discharged by `rhoctl id`. This catches a stale bitstream left in the fabric by a failed load.

Obligation P3 (Provenance)**.** A release bundle is built from a committed revision, and the simulation step passed on exactly that revision. Discharged by the build script, which refuses otherwise.

Obligation P4 (Log completeness)**.** Every log record the array produces reaches the file. Discharged by B8 together with a record count kept in the wrapper and compared with the count of records in the file at the end of a run.

Obligation P5 (Run reproducibility)**.** A run record names the bundle, the image and the seeds by digest, so the checker replays the same machine on the same input. Discharged by the run record format.

11 Bring-up ladder

Each rung has a pass condition; no rung is attempted before the one below it passes. The ladder is independent of the six stages of the design specification: L0 to L2 precede its stage 1 on hardware.

Rung	Content of the fabric	Pass condition
L0	Wrapper only: identity registers, `SCRATCH`, cycle counter	The whole path runs end to end; identity matches; `SCRATCH` reads back what was written; the cycle counter advances only while the run bit is set
L1	One tile, Boolean core	Directed programs of the design specification produce logs the checker accepts; utilisation and timing reports recorded
L2	2×2 array	As L1, with cross-tile programs; logs from the four tiles merge into a partial order the checker accepts
L3	10×10 array	As L2 at full size; if it does not fit or does not meet timing, the largest array that does, with the reports
L4	Builtin data types	Stages 2 onward of the design specification

L1 produces the first measured cost of a tile on this device. The board recommendation rested on an estimate of the fabric available per tile at 100 tiles; L1 replaces that estimate with a number, and L3 is planned from it.

12 Failure modes

Symptom	First thing to check
`loadapp` reports that an application is already loaded	Run `unloadapp` first
The application does not appear in `listapps`	Directory name, the shared base name of the bitstream and overlay, presence of `shell.json`
The load succeeds but `ID` reads zero or the bus hangs	Fabric clock not enabled by the overlay; base address in the overlay differs from the one Vivado assigned
Identity registers differ from the manifest	A previous bitstream is still loaded; the bundle on the target is not the one just built
The kernel warns of a memory leak when the overlay is removed	Reported as a known warning for overlays of this kind; record it and continue
The run stalls permanently	Log drain stopped; `STATUS` shows stalled; restart the drain
The board is unreachable by SSH	Serial console; read the network address again

13 Open questions

1. Which Linux image the target runs. Both images offered for this board provide `xmutil`; they differ in default user, package manager and how the device tree generator's output is expected to be laid out.

2. The Vivado version, which fixes the supported distributions for the build host and the exact form of the Tcl and of the device tree generation step.
3. Whether the kit as bought includes a power supply and a microSD card.
4. The fabric clock frequency, after L1.
5. Whether register-access log readout is fast enough at L3, or whether the streaming path of B9 is needed. With about 100 tiles each able to commit once per cycle, the log rate at full activity is far above what a register interface drains, so the array will spend most of its time stalled under B8 unless logging is made selectable per tile or the streaming path is built.
6. Whether the build host also keeps the image compiler, or whether images are compiled on the relay.
7. Rulings on B1 to B10.

14 Sources

The packaging and loading procedure follows the vendor's documentation and two published walk-throughs. None of it has been run for this project.

- Kria application documentation, "On-target utilities and firmware" and "Vivado accelerator flow example": https://xilinx.github.io/kria-apps-docs/creating_applications/2022.1/build/html/docs/target.html, https://xilinx.github.io/kria-apps-docs/creating_applications/2022.1/build/html/docs/vivado_accel_example.html
- Vitis platform creation tutorial for the KV260, step 4: https://xilinx.github.io/Vitis-Tutorials/master/docs-jp/docs/Vitis_Platform_Creation/Design_Tutorials/01-Edge-KV260/step4.html
- "Enabling custom PL overlays on KR260": https://controlpaths.pages.dev/2025/12/21/enabling-custom-pl-overlays-kr260
- "KV260 DPU TRD, PetaLinux 2022.1, Vivado flow": https://www.hackster.io/shreyasnr/kv260-dpu-trd-petalinux-2022-1-vivado-flow-000c0b
- Design specification, "A hardware rendering of the rho calculus with Boolean arithmetic and builtin data", draft 0.5, 13 November 2026.